

```
1: /**
2:  * @file restaurante.c
3:  * @brief Asignación recursiva de mesas para maximizar grupos atendidos
4:  * @author Asistente IA
5:  * @date 2026
6:  *
7:  * Este programa resuelve el problema de asignar grupos de clientes a mesas
8:  * de capacidad fija (4 personas) para maximizar la cantidad de grupos atendidos.
9:  */
10:
11: #include <stdio.h>
12:
13: /**
14:  * @brief Ordena un arreglo de enteros usando recursión (sin bucles)
15:  *
16:  * @param arr Arreglo a ordenar
17:  * @param n Tamaño del arreglo
18:  * @note Usa selección recursiva para evitar ciclos con break/continue
19:  */
20: void ordenarRecursivo(int *arr, int n) {
21:     // Caso base: arreglo de tamaño 0 o 1 ya está ordenado
22:     if (n <= 1) {
23:         return;
24:     }
25:
26:     // Encontrar el índice del mínimo elemento (sin usar break)
27:     int minIndex = 0;
28:     for (int i = 1; i < n; i++) {
29:         if (arr[i] < arr[minIndex]) {
30:             minIndex = i;
31:         }
32:         // No se usa break ni continue
33:     }
34:
35:     // Intercambiar el mínimo con el primer elemento
36:     if (minIndex != 0) {
37:         int temp = arr[0];
38:         arr[0] = arr[minIndex];
39:         arr[minIndex] = temp;
40:     }
41:
42:     // Ordenar recursivamente el resto del arreglo
43:     ordenarRecursivo(arr + 1, n - 1);
44: }
45:
46: /**
47:  * @brief Función recursiva principal para maximizar grupos atendidos
48:  *
49:  * @param grupos Arreglo con los tamaños de los grupos
50:  * @param n Número total de grupos
51:  * @param idx Índice actual en el arreglo de grupos
52:  * @param mesasDisponibles Número de mesas libres en este momento
53:  * @param capacidadMesa Capacidad máxima por mesa (4 personas)
54:  * @return int Máximo número de grupos que se pueden atender desde esta posición
55:  *
56:  * @details Esta función explora todas las combinaciones posibles usando:
57:  * - Si el grupo actual cabe y hay mesas: intenta asignarlo
58:  * - Siempre considera no asignar el grupo actual
59:  * - Toma la mejor opción entre ambas
60:  */
61: int maximizarGrupos(const int *grupos, int n, int idx,
62:                     int mesasDisponibles, int capacidadMesa) {
63:
64:     // === CASOS BASE ===
65:
66:     // Caso base 1: No quedan grupos por procesar
67:     if (idx >= n) {
68:         return 0;
69:     }
```

n

```
70:
71: // Caso base 2: No hay mesas disponibles
72: if (mesasDisponibles == 0) {
73:     return 0;
74: }
75:
76: // Caso base 3: El grupo actual es demasiado grande para la capacidad de la me
sa
77: if (grupos[idx] > capacidadMesa) {
78:     // Este grupo no se puede asignar, pasar al siguiente
79:     return maximizarGrupos(grupos, n, idx + 1,
80:                             mesasDisponibles, capacidadMesa);
81: }
82:
83: // === CASO RECURSIVO ===
84:
85: // Opci3n 1: NO asignar el grupo actual
86: // Simplemente avanzamos al siguiente grupo
87: int noAsignar = maximizarGrupos(grupos, n, idx + 1,
88:                                 mesasDisponibles, capacidadMesa);
89:
90: // Opci3n 2: ASIGNAR el grupo actual
91: // Requiere: grupo cabe y hay mesas disponibles
92: int asignar = 1 + maximizarGrupos(grupos, n, idx + 1,
93:                                   mesasDisponibles - 1, capacidadMesa);
94:
95: // Retornar la mejor opci3n (la que maximiza la cantidad de grupos)
96: if (asignar >= noAsignar) {
97:     return asignar;
98: } else {
99:     return noAsignar;
100: }
101: }
102:
103: /**
104:  * @brief Versi3n optimizada con poda recursiva
105:  *
106:  * @param grupos Arreglo ordenado de grupos (menor a mayor)
107:  * @param n N3mero total de grupos
108:  * @param idx 3ndice actual
109:  * @param mesasDisponibles Mesas libres
110:  * @param capacidadMesa Capacidad de cada mesa (4)
111:  * @param mejorActual Mejor resultado encontrado hasta ahora (para poda)
112:  * @return int M3ximo de grupos atendidos
113:  */
114: int maximizarGruposPoda(const int *grupos, int n, int idx,
115:                         int mesasDisponibles, int capacidadMesa,
116:                         int mejorActual) {
117:
118:     // Casos base (igual que antes)
119:     if (idx >= n || mesasDisponibles == 0) {
120:         return 0;
121:     }
122:
123:     // Poda: Si ya superamos el mejor resultado, no seguimos
124:     if (mejorActual > mesasDisponibles) {
125:         return 0;
126:     }
127:
128:     if (grupos[idx] > capacidadMesa) {
129:         return maximizarGruposPoda(grupos, n, idx + 1,
130:                                     mesasDisponibles, capacidadMesa,
131:                                     mejorActual);
132:     }
133:
134:     int noAsignar = maximizarGruposPoda(grupos, n, idx + 1,
135:                                         mesasDisponibles, capacidadMesa,
136:                                         mejorActual);
137:
138:     int asignar = 1 + maximizarGruposPoda(grupos, n, idx + 1,
```

```
139:                                     mesasDisponibles - 1,
140:                                     capacidadMesa, mejorActual);
141:
142:     if (asignar >= noAsignar) {
143:         return asignar;
144:     } else {
145:         return noAsignar;
146:     }
147: }
148:
149: /**
150:  * @brief Funci3n wrapper para resolver el problema
151:  *
152:  * @param grupos Arreglo con tamaÃ±os de grupos
153:  * @param numGrupos NÃºmero total de grupos
154:  * @param numMesas NÃºmero de mesas disponibles
155:  * @param capacidadMesa Capacidad mÃ¡xima por mesa (default 4)
156:  * @return int NÃºmero mÃ¡ximo de grupos que pueden ser atendidos
157:  */
158: int resolverRestaurante(int *grupos, int numGrupos,
159:                         int numMesas, int capacidadMesa) {
160:
161:     // Paso 1: Ordenar grupos de menor a mayor (estrategia greedy para optimizar)
162:     // Ordenamos porque es mejor asignar grupos pequeÃ±os primero
163:     ordenarRecursivo(grupos, numGrupos);
164:
165:     printf("Grupos ordenados: ");
166:     for (int i = 0; i < numGrupos; i++) {
167:         printf("%d ", grupos[i]);
168:     }
169:     printf("\n");
170:
171:     // Paso 2: Llamar a la funci3n recursiva
172:     return maximizarGrupos(grupos, numGrupos, 0,
173:                           numMesas, capacidadMesa);
174: }
175:
176: /**
177:  * @brief Versi3n con seguimiento detallado para depuraci3n
178:  *
179:  * @param grupos Arreglo ordenado de grupos
180:  * @param n Total de grupos
181:  * @param idx 215ndice actual
182:  * @param mesasDisponibles Mesas libres
183:  * @param capacidadMesa Capacidad de la mesa
184:  * @param profundidad Nivel de recursi3n (para indentaci3n)
185:  * @return int MÃ¡ximo de grupos atendidos
186:  */
187: int maximizarGruposDebug(const int *grupos, int n, int idx,
188:                         int mesasDisponibles, int capacidadMesa,
189:                         int profundidad) {
190:
191:     // Imprimir indentaci3n
192:     for (int i = 0; i < profundidad; i++) {
193:         printf(" ");
194:     }
195:
196:     printf("Evaluando grupo[%d]=%d (mesas=%d)\n",
197:           idx, idx < n ? grupos[idx] : -1, mesasDisponibles);
198:
199:     // Casos base
200:     if (idx >= n) {
201:         printf("%sâ\206\222 No hay mÃ¡s grupos, retorna 0\n", profundidad * 2, ""
);
202:         return 0;
203:     }
204:
205:     if (mesasDisponibles == 0) {
206:         printf("%sâ\206\222 No hay mesas disponibles, retorna 0\n", profundidad *
2, "");
```

```
207:     return 0;
208: }
209:
210: if (grupos[idx] > capacidadMesa) {
211:     printf("%sâ\206\222 Grupo %d excede capacidad (%d > %d), se omite\n",
212:         profundidad * 2, "", grupos[idx], grupos[idx], capacidadMesa);
213:     return maximizarGruposDebug(grupos, n, idx + 1,
214:         mesasDisponibles, capacidadMesa,
215:         profundidad + 1);
216: }
217:
218: // OpciÃ³n 1: No asignar
219: printf("%s--- OpciÃ³n: NO asignar grupo %d ---\n",
220:     profundidad * 2, "", grupos[idx]);
221: int noAsignar = maximizarGruposDebug(grupos, n, idx + 1,
222:     mesasDisponibles, capacidadMesa,
223:     profundidad + 1);
224:
225: // OpciÃ³n 2: Asignar
226: printf("%s--- OpciÃ³n: ASIGNAR grupo %d (usa 1 mesa) ---\n",
227:     profundidad * 2, "", grupos[idx]);
228: int asignar = 1 + maximizarGruposDebug(grupos, n, idx + 1,
229:     mesasDisponibles - 1,
230:     capacidadMesa, profundidad + 1);
231:
232: // Comparar resultados
233: int resultado;
234: if (asignar >= noAsignar) {
235:     resultado = asignar;
236:     printf("%sâ\234\223 Mejor: ASIGNAR (%d > %d)\n",
237:         profundidad * 2, "", asignar, noAsignar);
238: } else {
239:     resultado = noAsignar;
240:     printf("%sâ\234\223 Mejor: NO ASIGNAR (%d > %d)\n",
241:         profundidad * 2, "", noAsignar, asignar);
242: }
243:
244: return resultado;
245: }
246:
247: /**
248:  * @brief FunciÃ³n principal del programa
249:  */
250: int main() {
251:     // === CASO DE PRUEBA PROPORCIONADO ===
252:     printf("=== PROBLEMA DE ASIGNACIÃ³N DE MESAS ===\n");
253:     printf("Capacidad por mesa: 4 personas\n\n");
254:
255:     int numMesas = 6;
256:     int gruposInicial[] = {2, 3, 4, 1, 5};
257:     int numGrupos = sizeof(gruposInicial) / sizeof(gruposInicial[0]);
258:
259:     printf("Entrada:\n");
260:     printf("  Mesas disponibles: %d\n", numMesas);
261:     printf("  Grupos: ");
262:     for (int i = 0; i < numGrupos; i++) {
263:         printf("%d ", gruposInicial[i]);
264:     }
265:     printf("\n\n");
266:
267:     // Copiar arreglo para no modificar el original
268:     int grupos[100];
269:     for (int i = 0; i < numGrupos; i++) {
270:         grupos[i] = gruposInicial[i];
271:     }
272:
273:     // Resolver el problema
274:     int resultado = resolverRestaurante(grupos, numGrupos, numMesas, 4);
275:
276:     printf("\nResultado:\n");
```

```
277: printf(" Máximo de grupos atendidos: %d\n", resultado);
278: printf(" (Esperado: 4)\n\n");
279:
280: // === VERSIÃ\223N CON DEPURACIÃ\223N ===
281: printf("=== PROCESO DETALLADO DE RECURSIÃ\223N ===\n");
282:
283: // Reordenar grupos nuevamente para debug
284: int gruposDebug[] = {2, 3, 4, 1, 5};
285: ordenarRecursivo(gruposDebug, numGrupos);
286:
287: printf("Ã\201rbol de decisiÃ³n (DFS):\n");
288: int resultadoDebug = maximizarGruposDebug(gruposDebug, numGrupos, 0,
289:                                           numMesas, 4, 0);
290:
291: printf("\nResultado final: %d grupos atendidos\n", resultadoDebug);
292:
293: // === PRUEBAS ADICIONALES ===
294: printf("\n=== PRUEBAS ADICIONALES ===\n");
295:
296: // Prueba 1: Todos los grupos caben
297: int prueba1[] = {1, 2, 1, 2};
298: int grupos1[4];
299: for (int i = 0; i < 4; i++) grupos1[i] = prueba1[i];
300: printf("\nPrueba 1 - Grupos [1,2,1,2], Mesas=4:\n");
301: printf(" Resultado: %d\n", resolverRestaurante(grupos1, 4, 4, 4));
302: printf(" Esperado: 4 (todos caben)\n");
303:
304: // Prueba 2: Grupos muy grandes
305: int prueba2[] = {5, 6, 4, 3};
306: int grupos2[4];
307: for (int i = 0; i < 4; i++) grupos2[i] = prueba2[i];
308: printf("\nPrueba 2 - Grupos [5,6,4,3], Mesas=3:\n");
309: printf(" Resultado: %d\n", resolverRestaurante(grupos2, 4, 3, 4));
310: printf(" Esperado: 2 (solo 4 y 3, o 3 y 4)\n");
311:
312: // Prueba 3: Sin mesas
313: int prueba3[] = {1, 2, 3};
314: int grupos3[3];
315: for (int i = 0; i < 3; i++) grupos3[i] = prueba3[i];
316: printf("\nPrueba 3 - Grupos [1,2,3], Mesas=0:\n");
317: printf(" Resultado: %d\n", resolverRestaurante(grupos3, 3, 0, 4));
318: printf(" Esperado: 0\n");
319:
320: return 0;
321: }
```